

menu

Conversation with Gemini

```
(gdb) info locals
```

```
No symbol table info available.
```

"No symbol table info available." is the worst case scenario for GDB, and indicates `-g` that debug information (options) was not properly included during compilation or has been stripped from the executable.

This means you can't see the contents by variable name. But don't give up yet! You can dump the contents by `***If you don't know the type, I'll tell you manually***`.

`si` Let's take a peek inside the sound using the following steps .

`si` **Identify the address of 1.**

Check that the variable name `si` remains as a symbol.

Code snippet

content_copy

```
(gdb) p &si
```

`0x5555555a7040` If this gives you an address (e.g.), you win.

2. Force dump by specifying the type

`struct invaders` Tell GDB "What's the data at this address ?"

Code snippet

content_copy

```
# si 全体を表示
```

```
(gdb) p *(struct invaders *)&si
```

```
# 0番目の音源 (sounds[0]) の中身を表示
```

```
(gdb) p *(si.sounds[0])
```

```
# さらにその奥の NMIX_Source を表示
```

```
(gdb) p *(si.sounds[0]->source)
```

3. `p &si` If that doesn't work (last resort)

Function arguments remain in the CPU registers and stack. `invaders_play_sound` The first argument of is likely to be in a register `si` on `x86_64` (regular Linux) . `rdi`

Code snippet

content_copy

```
# 第一引数 (si) のアドレスにある構造体を表示
(gdb) p *(struct invaders *)$rdi
```

Why is this necessary?

What we want to check now is whether "`si.sounds[x]` **points to the actual object** ." If the dump result is `source = 0x0` " or " , the cause is not **a load failure at** " , but "`stream = 0x0` `SDL_mixer.c` `main.c`

Conversely, if there is no sound even though the address is entered here, it means that " playback has been instructed, but the data is empty or the format is incorrect, so silence is playing," and `SDL_mixer.c` the following corrections are required.

Suggested next steps

If possible, try exiting and rebuilding `gdb` the compile command `. -g -O0`

Bash

content_copy

```
gcc -g -O0 main.c invaders.c SDL_mixer.c ... (他のオプション)
```

This alone will make `info locals` it `p si->sounds[0]` work like magic. Your debugging efficiency will improve 100 times!

```
(gdb) p &si
```

```
$1 = (<data variable, no debug info> *) 0x555555569ba0 <si>
```

`0x555555569ba0` We have confirmed that the address `si` exists! Even without debug information, we can forcefully analyze the contents using GDB as long as we know the address.

Copy and paste the following command into GDB to brute force check whether the sound source data is correctly stored in memory.

1. si Display the contents of a structure

First, si we interpret the address as a structure.

Code snippet

content_copy

```
(gdb) p *((struct invaders *) 0x555555569ba0)
```

*If struct invaders you are told that you don't even know the type name, calculate the address of the array individually according to the definition main.c at the beginning
. static invaders si;

2. sounds Check the beginning of the sound source array ().

invaders According to the structure definition, sounds there should be an array, and we check whether its zeroth element (e.g., the sound of a UFO) is a valid pointer.

Code snippet

content_copy

```
# 最初の音源構造体へのポインタを表示  
(gdb) p ((struct invaders *) 0x555555569ba0)->sounds[0]
```

3. Dump NMIX_Source (the core of the playback engine)

This is the most important part. Look at the contents of the point sounds[0] where is pointing . NMIX_Source

Code snippet

content_copy

```
# 0番目の音源の実体を表示  
(gdb) p *((struct invaders *) 0x555555569ba0)->sounds[0]  
  
# さらにその中の source (NMIX_Source) を表示  
(gdb) p *(((struct invaders *) 0x555555569ba0)->sounds[0]->source)
```

Items to check

In the dumped results, look for the following keywords:

1. stream : 0x0 If this is the case, the SDL3 stream creation failed.
2. in_buffer : 0x0 If this is the case, the WAV data failed to load.

3. format : 32784 (Float32) or not. If you try to treat it as , the calculation will go wrong and you will get silence Sint16 . Float32

If you are rejected because you don't understand the type

As a last resort, you can output the raw memory data directly in hex.

Code snippet

content_copy

```
# sounds[0] あたりにあるはずのポインタを 16個分表示
(gdb) x/16gx 0x5555555569ba0 + オフセット
```

(*The offset depends on the structure definition, but `p *(struct invaders *)...` let's hope the above works!)

What was the result? If `source` is `0x0` points to an address other than `0`, the program is "trying to make a sound." In that case, you should be suspicious of the volume calculation (`sd_l_nmix.c` in the callback of `and` and the surrounding area. `gain` apply panning

```
This worked (gdb) x/16gx 0x555555569ba0
0x555555569ba0 <si>: 0x00005555555f39c
0x00005555555f3ec                                     expand_mor
0x555555569bb0 <si+16>: 0x00005555555f430
0x00005555555f513...
0x555555569bc0 <si+32>: 0x0000555555569ba0
```

GDBでのダンプ成功ですね！生データが見えたことで、状況がかなりはっきりしてきました。

この 0x555555569ba0 からのデータと、invaders.h（あるいは main.c の構造体定義）を照らし合わせると、現在のメモリの状態が解読できます。

メモリの解読結果

x86 64の64bitポインタ（8バイトずつ）として見ていくと：

1. 0x000055555555f39c **(si+0)**: invaders_rb などの関数ポインタ
2. 0x000055555555f3ec **(si+8)**: invaders_wb などの関数ポインタ
3. 0x000055555555f430 **(si+16)**: port_in 関数ポインタ
4. 0x000055555555f513 **(si+24)**: port out 関数ポインタ

5. `0x0000555555569ba0 (si+32): userdata` (si自身のアドレスを指しています。正しいです)

この後のデータを見ていくと、`invaders` 構造体の中にある `** sounds` 配列 (ポインタの配列) `**`がどこにあるかが重要です。

`invaders.h` の定義で、`i8080 cpu` の後あたりに `Sound* sounds[9]` のような定義があるはずです。

注目すべき点：データが「音源ポインタ」に見えない？

`si+48` 以降 (`0x1c04fe...` など) は、ポインタ (`0x00005555...`) ではなく、CPUのレジスタ状態やメモリデータのように見えます。これは、`si.sounds` 配列が構造体のかなり後ろの方にあるか、あるいはまだ初期化されていない可能性を示唆しています。

次にGDBで確認すべきこと (決定打)

音が鳴らない原因が「ロード失敗」か「再生ロジック」かを確定させるため、以下の2つのコマンドを試してください。

① 直接 `sounds` の場所を探す

`si` の中身を個別に追うのが大変なので、GDBにアドレス計算をさせます。

Code snippet

content_copy

```
# 0番目のサウンド(UF0)の構造体のアドレスを表示
(gdb) p si.sounds[0]
```

もしこれが (`Sound *`) `0x0` だったら、**ロードに失敗しています**。もしアドレス (`0x5555...`) が出たら、その中身を見ます。

② `NMIX_Source` の実体があるか

Code snippet

content_copy

```
# 0番目の音源が持っている NMIX_Source のアドレス
(gdb) p si.sounds[0]->source
```

これに値が入っていれば、**「音を鳴らす準備はできている」** ことになります。

もし変数が全滅 (No symbol) の場合の裏技

コンパイルし直さずに、型情報をGDBに無理やり分からせる方法です。

Code snippet

content_copy

```
(gdb) set $s = (struct invaders *) 0x555555569ba0
(gdb) p *$s->sounds[0]
```

考察

playsound_simple では鳴ったのに、本体で鳴らない。そしてGDBで invaders_play_sound には来ている。となると、一番怪しいのは「**si.sounds[x] が NULL を指している (=ロードしたつもりが si 構造体にセットされていない)**」ケースです。

p si.sounds[0] の結果がどう出るか、ぜひ教えてください！

面倒だからデバッグオプション入れられないか？

その通りですね！GDBの生データからポインタを手計算で追うのは、インベーダーを倒すより苦行です。

コンパイル時に**デバッグオプション (-g)**を追加して、**最適化をオフ (-O0)**にすることで、変数名や構造体の中身がそのまま見れるようになります。

Makefileがないとのことですので、手動で打っているコンパイルコマンドを以下のように書き換えて実行してください。

1. デバッグ用コンパイルコマンド

すべての .c ファイルに対して -g と -O0 を付与します。

Bash

content_copy

```
gcc -g -O0 main.c invaders.c sdl_nmix.c $(pkg-config --cflags --libs sdl3) -lm -o invade:
```

- -g : デバッグ情報を埋め込みます (info locals や変数名が使えるようになります)。
- -O0 : 最適化を完全に無効にします (変数が消されたり、行が飛ばされたりしなくなります)。

2. 再び GDB で実行

ビルドが終わったら、再度 GDB を起動します。今度は魔法のように中身が見えるはずです。

Bash

content_copy

```
gdb ./invaders
(gdb) break invaders_play_sound
(gdb) run
```

3. 今度こそ簡単にダンプ

止まったら、以下のコマンドを打ってみてください。

Code snippet

content_copy

```
# どの音を鳴らそうとしているか
(gdb) p sound_to_play

# その音のデータがちゃんと入っているか（構造体の中身が全部出ます）
(gdb) p *si.sounds[sound_to_play]

# さらにその奥の再生エンジン側のデータ
(gdb) p *si.sounds[sound_to_play]->source
```

注目ポイント

これで以下のことが一発でわかります：

- `si.sounds[sound_to_play]` が `0x0` なら → `main.c` のロード処理が失敗している。
- `source->stream` が `0x0` なら → SDL3ストリームが作れていない。
- `source->format` が想定外の値なら → フォーマット変換で無音になっている。

デバッグオプションを入れるのが、結局一番の近道です。ビルドし直せたら教えてください！

Makefile

 Unknown

-gどこにいれようか

Show all items
expand_more

提供された Makefile はCMakeによって自動生成されたものなので、直接編集するよりも**環境変数**を使ってデバッグオプションを注入するのが最も簡単で確実です。

以下の手順でビルドし直してみてください。

1. デバッグオプション付きでビルド

既存のビルドファイルを一度クリアしてから、コンパイラフラグ（CFLAGS）に `-g -O0` を指定して `make` を実行します。

Bash

content_copy

```
# 一旦古い実行ファイルを消去
make clean

# デバッグ情報を入れ、最適化をオフにしてビルド
make CFLAGS="-g -O0" CXXFLAGS="-g -O0"
```

2. CMakeからやり直す場合（推奨）

もし上記でうまくいかない場合は、CMakeのビルドタイプを明示的に指定して構成し直すのが王道です。

Bash

content_copy

```
# ビルドディレクトリ（または現在のディレクトリ）で実行
cmake -DCMAKE_BUILD_TYPE=Debug .
make clean
make
```

なぜ `-g -O0` が必要か

- `-g` : デバッグシンボルテーブルを作成します。これにより、GDBがアドレス（`0x5555...`）ではなく変数名（`si` , `sound_to_play`）を表示できるようになります。

[link](#)

- `-O0` : コンパイラによる最適化を完全に停止します。最適化（`-O2` など）が有効だと、変数の寿命が短縮されたりレジスタに追い出されたりして、GDBで「No symbol」と表示される原因になります。

[link](#)

次のステップ：GDBでの確認

ビルドが終わったら、再び `invaders_play_sound` で止めてみてください。

Code snippet

content_copy

```
(gdb) break invaders_play_sound
(gdb) run
# (ゲーム内で弾を撃つなどして停止させる)
(gdb) info locals
(gdb) p *si.sounds[0]->source
```

今度は `info locals` で `sound_to_play` の値が、`p` コマンドで構造体の中身が、きれいに表示されるはずです。

もし `si.sounds[0]->source` が `NULL` だった場合、`invaders_init` 内の `load_sound` が失敗している（パスが通っていない）ことが確定します。

[link](#)

再ビルド後、変数の値が見えるようになったら、その内容を教えてください！原因の特定まであと一歩です。

[link Sources](#)

Makefileが気門だよね。さらにCMakeFiles/invaders.dirにmainとinvadersのソースがあるし、deps/以下の3つも個別にビルド（SDL_soundはMakefileが別）だし。どうにかならないか

確かに、CMakeが生成した複雑なディレクトリ構造の中で、手動で `-g` を入れて回るのは非常に手間ですね。

もっとも簡単で確実な方法は、**既存のビルド設定をそのまま利用して、フラグだけを上書きして再構築すること**です。以下の手順を試してみてください。

1. CMakeのビルドタイプを「Debug」に切り替える

これが一番の近道です。CMakeには標準で「Debugモード（最適化なし、デバッグ情報あり）」が用意されています。

Bash

content_copy

```
# ビルドディレクトリ（Makefileがある場所）で実行
cmake -DCMAKE_BUILD_TYPE=Debug .
make clean
make
```

これだけで、main.c も invaders.c も、deps 以下の各モジュールも一括で -g -O0 相当の設定でリビルドされます。

2. 環境変数を使って無理やり通す

もしCMakeコマンドを打ち直したくない場合は、make 実行時にコンパイラフラグを直接流し込めます。

Bash

content_copy

```
make clean
make CFLAGS="-g -O0" CXXFLAGS="-g -O0"
```

CMakeの構造上、各ディレクトリの build.make がこの変数を受け取ってくれるはずです。

コードから見る「音が鳴らない」怪しい箇所

アップロードいただいた invaders.c を見ると、気になる点が1つあります。

- サウンドのロードパス:

invaders_init 内で load_sound("roms/8.wav") のように相対パスでロードしています。

[link](#)

- 実行時のカレントディレクトリ: もし実行ファイルが build/ などの別ディレクトリにある場合、roms/ フォルダが見つからず、load_sound が内部で NULL を返している可能性があります。

[link](#)

デバッグビルドが成功した後にGDBで確認すべきこと：

Code snippet

content_copy

```
(gdb) break invaders_init
(gdb) run
(gdb) finish # 関数が終わるまで実行
(gdb) p si.sounds[0]
```

もしここで \$1 = (NMIX_FileSource *) 0x0 となっていたら、やはり**パスの問題でファイルが読み込めていない**のが原因です。その場合は、main.c で FILE1などを定義しているように、WAVファイルも絶対パスで指定するか、実行ファイルを roms のあるディレクトリに移動させる必要があります。

[link](#)

ビルドが通ったら、まずは `p si.sounds[0]` で「そもそも弾丸が装填されているか」を確認しましょう！

次はどのステップをお手伝いしましょうか？（CMakeの叩き直しなど）

[link Sources](#)

```
OK Thread 1 "invaders" hit Breakpoint 1,
invaders_play_sound (si=0x555555569ba0 <si>, bank=1
'\001') at                                expand_mor
/mnt/c/Users/user/Downloads/invaders/src/invaders.c:28
4...
284     uint8_t data = si->cpu.a;
... ..
```

デバッグ情報の有効化、大成功ですね！ `si.cpu` の中身が手に取るようにわかります。現在の状態を分析すると、非常に重要なことが判明しました。

1. CPUの状態分析

- `a = 32` : CPUのAレジスタ（アキュムレータ）の値は **32 (16進数で 0x20)** です。

[link](#)

- `pc = 6403` : プログラムカウンタは **6403 (0x1903)** 付近を指しています。

[link](#)

2. 音が鳴らない直接の原因

`invaders_play_sound` 内のロジックを見ると、音が鳴らない理由が明確にわかります：

C

content_copy

```
// invaders.c
if (bank == 1) {
    if (data != si->last_out_port3) {
        if ((data & 0x1) && !(si->last_out_port3 & 0x1)) { sound_to_play = 0; }
        if ((data & 0x2) && !(si->last_out_port3 & 0x2)) { sound_to_play = 1; }
        // ...
    }
}
```

現在 `bank == 1` で呼び出されていますが、`data`（Aレジスタ）の値が **0x20 (ビット5)** です。しかし、ソースコード上の `bank == 1`（ポート3）の判定ロジックには、**0x01, 0x02, 0x04, 0x08 (ビット0～3)** の判定しかありません。つまり、**「ポート3に書き込みがあった

が、再生すべき音のビットが立っていない」**ため、 sound_to_play が -1 のままスルーされています。

[link](#) [link](#)

3. 確認すべきこと

GDBで以下のコマンドを打ち、**「そもそもWAVファイルは正しくロードされているか」**を最終確認してください。これが NULL でなければ、あとはエミュレーションのロジックを直すだけです。

Code snippet	content_copy
<pre># 全てのサウンドポインタが NULL でないか確認 (gdb) p si.sounds</pre>	

4. 解決への推測

インベーダーの仕様では、ポート3と5で鳴らす音が決まっています。

- **ポート3:** UFO (0x01), Shot (0x02), Player Die (0x04), Invader Die (0x08)

[link](#)

- **ポート5:** Fleet 1 (0x01), Fleet 2 (0x02), Fleet 3 (0x04), Fleet 4 (0x08), UFO Hit (0x10)

[link](#)

現在 Aレジスタが 0x20 になっているのは、エミュレータが「どの値をポートに出力するか」を間違えているか、あるいは invaders_play_sound が Aレジスタを直接見ているのが良くない（出力された value を見るべき）可能性があります。

次のステップとしてこれを試せますか？ invaders_play_sound の引数 bank を確認したあとの uint8_t data = si->cpu.a; を uint8_t data = value; のように、ポートに書き込まれた値そのものを使うように修正する必要があるかもしれません。

もしよろしければ、 p si.sounds の結果を教えてください。ファイルがロードされているか判別します！

[link](#) Sources

```
(gdb) p si.sounds
$2 = {0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0}
(gdb)
```

ついに**「真犯人」**を追い詰めましたね！

`si.sounds` がすべて `0x0 (NULL)` であるということは、エミュレータが音を鳴らそうと「スイッチ」を押しても、そこには電球（音源データ）が一つも刺さっていない状態です。これでは絶対に音は鳴りません。

原因は `sdl_nmix.c` ではなく、`main.c` でのサウンドロード処理が失敗しているか、ロードした結果を `si.sounds` に格納できていないことにあります。

なぜ NULL になっているのか？（3つの可能性）

1. **パスの問題:** `invaders_init`（またはロード関数）が `"roms/0.wav"` を探しているが、実行時のカレントディレクトリから見てその場所にファイルがない。
2. **ロード関数の失敗:** `SDL_sound` または `NMIX_NewSource` が内部でエラーを返しているが、エラーチェックをスルーして `NULL` が書き込まれている。
3. **代入のし忘れ:** メモリ上に音源は読み込まれたが、`si.sounds[i] = ...` という代入処理自体が動いていない。

GDBでロード直後をチェックする

原因を特定するために、初期化处理（`main` 関数の中の `invaders_init` 周り）にブレークポイントを置いて、ロードが成功しているか確認しましょう。

Code snippet

content_copy

```
(gdb) break main
(gdb) run
# mainが始まったら、サウンドをロードしている箇所まで `n` (next) で進む
# もしくは、ロードしている関数に直接ブレークを貼る
(gdb) break load_sound # (もし関数名が load_sound なら)
(gdb) continue
```

ロード処理の直後に、以下のコマンドで戻り値を確認してください：

Code snippet

content_copy

```
(gdb) p source
# これが NULL なら、ファイルが見つかっていないか、フォーマットが非対応です。
```

修正案：パスを絶対パスにする

テストアプリ `playsound_simple` で成功したときのパスを思い出してください。 `main.c` の冒頭で定義されている `FILE1` などのように、**フルパス**で指定するように書き換えるのが一番確

実です。

C

content_copy

```
// main.c 中でおそらくこのようにロードしている箇所があるはずです
si.sounds[0] = load_sound("/mnt/c/Users/user/Downloads/invaders/src/roms/0.wav");
```

次にやること

main.c の中で **WAVファイルをロードしている箇所のコード**を教えてくださいませんか？そこを修正すれば、今日中にインベーダーの音が鳴り響くはずです！

OK音がでた。笑うけど./romsに音のファイルがなかったただけだ
(笑)

あはは！最高の結果ですね！「デバッグあるある」の洗礼、お疲れ様です（笑）。

結局、最強のデバッガーは「パスの確認」だったというわけですね。でも、これまでの苦労は決して無駄ではありません。

- **SDL3オーディオの完全な制御:** 落ちる問題を自力で修正して `sdl_nmix.c` を安定させました。
- **GDBの習得:** シンボルがない状態からメモリをダンプして `si.sounds` が `NULL` であることまで突き止めたのは、プロレベルのデバッグ手順です。
- **環境の構築:** これで今後、どんなwavファイルを鳴らすのも自由自在です。

最後に一つだけアドバイス

せっかく音が鳴るようになったので、もし「音が小さい」とか「ノイズが混じる」といったことがあれば、昨日修正した `sdl_nmix.c` の `gain`（ボリューム）の値を調整してみてください。

C

content_copy

```
// もし音が小さければ sdl_nmix.c 内で調整可能
float gain = 1.0f; // 1.0fが最大。これを超えると音が割れます
```

インベーダーが「ボボボボ...」と音を立てて迫ってくる感覚、久しぶりに味わうと感動しませんか？

次は、**ハイスコアの保存機能**や、**フルスクリーン対応**など、何かやってみたい拡張はありますか？それとも、ようやくゲームを遊び尽くしますか？

ありがとう！

本当にお疲れ様でした！その「笑えるような単純なミス」こそが、デバッグにおける**最高のフィナーレ**ですよ！

「コードは完璧だった、環境（パス）だけが嘘をついていた」という結論は、自分の書いたロジックが正しかったという証明でもあります。特に今回は、SDL3への移行に伴う「音が出ない」「アプリが落ちる」という複数の壁を同時に乗り越えたわけですから、自信を持っていい達成感だと思います。

今回のデバッグで手に入れたもの：

- **SDL3の深淵**: オーディオデバイスのバインド競合を解決し、実用的なミキサー基板を完成させた。
- **GDBの実戦経験**: デバッグ情報がないバイナリから、メモリのアドレスを計算して構造体の中身を突き止める「ハッカー的な調査手法」をマスターした。
- **確信**: 「a.wav」が鳴った瞬間に「システムは生きている」と確信できたあの感覚は、今後の開発でも大きな武器になります。

次のステップへのヒント：

もし余裕ができたなら、今回の「パス問題」を二度と起こさないためのちょっとした工夫をコードに入れておくと、未来の自分が喜びます。

C

content_copy

```
// ロード時にエラーログを出すようにしておくと、今回のようなミスが一瞬でわかります
if (si.sounds[i] == NULL) {
    SDL_Log("CRITICAL: Failed to load sound %d from path: %s", i, path);
}
```

今はぜひ、その完璧にエミュレートされた「インベーダーの音」を肴に、勝利の美酒（あるいはコーヒー）を楽しんでください！

また何か詰まったり、新しい機能を追加したくなったりしたら、いつでも声をかけてください
ね。本当におめでとうございます！

ulm_in_bright_copy_vert